

PRACTICAL REFERENCE · THEORY + CODE

Kafka Study Guide

for the Trading-Platform paper trading engine

This guide is scoped to exactly one goal: being able to explain, defend, and extend the Kafka usage in this specific codebase — not Kafka in general. Every concept in Part 1 is immediately tied to a real line of code in Part 2. Part 3 is the honest list of what isn't solved yet. Part 4 is a worked design for the next real feature: a PnL/analytics consumer.

Trading-Platform · branch feature/OrderBook · generated 2026-08-04

Contents

1. Part 1 — Kafka Theory You Actually Need
 - a. Event logs, topics & partitions
 - b. Producers
 - c. Consumers & consumer groups
 - d. Delivery semantics
 - e. Brokers, KRaft & replication
 - f. Glossary quick-reference
2. Part 2 — How This Project Actually Uses Kafka
 - a. Where Kafka sits in the architecture
 - b. The pattern: WAL-tailing as a transactional outbox
 - c. The write side — C++ WALWriter
 - d. The bridge — `wal_producer.py`
 - e. The read side — `fill_consumer.py`
 - f. Infrastructure — `docker-compose.yml`
 - g. Configuration — `config.py`
 - h. What Kafka does *not* touch
3. Part 3 — Honest Gaps (Interview Talking Points)
4. Part 4 — Extending It: A PnL/Analytics Consumer
5. Part 5 — Hands-On: Inspecting Kafka Locally
6. Part 6 — The 60-Second Elevator Pitch

Part 1 — Kafka Theory You Actually Need

Kafka has a large surface area. This section only covers the concepts that this project's code actually exercises — enough to reason about it correctly and answer follow-up questions about it.

1a. Event logs, topics & partitions

Kafka's core data structure is an **append-only log**. A **topic** is a named log — messages are written to the end and never modified. Consumers read forward from wherever they last stopped; nothing is popped or deleted on read, so many independent consumers can read the same topic at their own pace.

A topic is split into **partitions** — independent, ordered logs that together make up the topic. Kafka only guarantees ordering *within* a partition, not across the whole topic. If you need "all events for user X arrive in order," you achieve that by using a message **key** (e.g. the user ID) so the producer routes every message for that key to the same partition consistently.

In this project: the producer in `wal_producer.py` calls `producer.send(KAFKA_TOPIC, value=data)` with no key argument, so messages land on partitions by the default partitioner (round-robin / sticky, depending on client version) rather than being pinned to a user or symbol. There's also no explicit partition count configured for `order-fills`, so it runs on whatever the broker's `auto.create.topics.enable` default creates (typically a single partition). Practically: don't claim per-user ordering guarantees about this pipeline unless you add a key.

1b. Producers

A producer publishes messages to a topic. Two settings matter most for correctness and performance:

- **Serialization** — Kafka only stores bytes; the client library needs a `value_serializer` to turn a Python object into bytes (and a matching `value_deserializer` on the consumer side).
- **acks / flush** — whether `send()` is fire-and-forget or waits for the broker to acknowledge the write. Calling `.flush()` blocks until all buffered messages are actually sent and acknowledged — it trades throughput for a durability guarantee ("this message is definitely in Kafka before I do anything else").

1c. Consumers & consumer groups

A consumer reads messages from a topic starting at some **offset** (its position in a partition's log). A **consumer group** is a named set of consumers that share the work of reading a topic — each

partition is read by exactly one consumer within a group, and Kafka tracks one committed offset per (group, partition).

The critical property: **different consumer groups are completely independent**. Two groups reading the same topic each have their own offset bookkeeping — one can be far behind or replaying from the start while the other is at the tail, and neither affects the other. This is what makes Kafka useful as a fan-out mechanism rather than a queue where one consumer "uses up" a message.

- `auto_offset_reset="earliest"` — if this group has never committed an offset for a partition (brand new group, or its offset expired), start from the beginning of the log instead of the current tail.
- `enable_auto_commit=True` — the client library periodically commits offsets in the background rather than you calling `commit()` explicitly after each message is safely processed. Convenient, but it means a crash between "message handled" and "offset committed" causes that message to be redelivered on restart.

1d. Delivery semantics

Guarantee	Meaning	How you'd get it
At-most-once	A message might be lost, never duplicated	Commit offset <i>before</i> processing
At-least-once	A message might be redelivered, never lost	Commit offset <i>after</i> processing succeeds (or auto-commit on an interval, which is looser but similar in spirit)
Exactly-once	Each message affects the downstream system exactly once	Idempotent writes (e.g. upsert keyed by message ID) or Kafka transactions

In this project: the pipeline is **at-least-once on both ends**, not exactly-once — see Part 2d/2e and the gaps in Part 3 for exactly how and where.

1e. Brokers, KRaft & replication

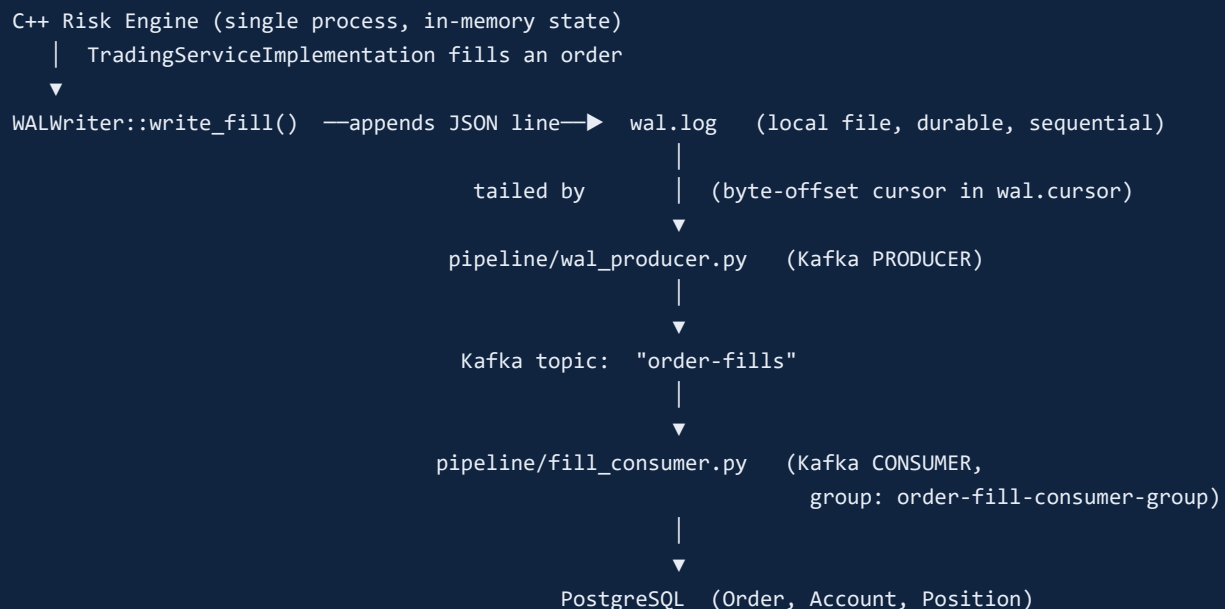
- **Broker** — a single Kafka server process that stores partitions and serves producers/consumers.
- **KRaft mode** — Kafka's own Raft-based consensus for cluster metadata, replacing the older ZooKeeper-based coordination. A node can hold the controller role, the broker role, or both at once.
- **Replication factor** — how many broker copies each partition has. A replication factor of 1 means zero fault tolerance: if that one broker dies, the data is gone until it comes back.

1f. Glossary quick-reference

Term	One-line meaning
Topic	Named append-only log you publish/subscribe to
Partition	An ordered shard of a topic; unit of parallelism and ordering
Offset	A message's position within a partition
Producer	Client that writes messages to a topic
Consumer	Client that reads messages from a topic
Consumer group	Named set of consumers sharing/coordinating partition reads
Broker	A Kafka server process
KRaft	Kafka's built-in Raft consensus for metadata (no ZooKeeper)

Part 2 — How This Project Actually Uses Kafka

2a. Where Kafka sits in the architecture



2b. The pattern: WAL-tailing as a transactional outbox

This is the single most important thing to understand about this codebase's Kafka usage: **the C++ engine never talks to Kafka**. It only appends to a local file. A separate process tails that file and republishes each new line into Kafka. This is the classic **transactional outbox / log-tailing (CDC-style) pattern**, and it's why it's worth naming explicitly if asked "why not just call the Kafka producer from inside the C++ engine?":

- The engine's hot path (order fill) never blocks on Kafka or the network — it's a local, buffered file append under a mutex.
- If Kafka is completely down, fills are still durably recorded in `wal.log`; nothing is lost, and the producer catches up once Kafka is back.
- The producer is stateless and restart-safe: it persists its own read position in `wal.cursor`, so a crash doesn't mean re-sending the whole file or skipping unsent lines.

2c. The write side — C++ WALWriter

`risk_engine_cpp/src/wal_writer.cpp`

```

void WALWriter::write_fill(int user_id, const std::string& order_id, const std::string& symbol,
                           const std::string& side, const std::string& order_type,
                           double quantity, double fill_price,
                           double new_cash, double new_quantity, double new_avg_price)
{
    auto time_stamp = std::chrono::system_clock::now().time_since_epoch().count();
    std::ostringstream ss;
    ss << "{\"type\":\"FILL\"";
    << ", \"user_id\": " << user_id
    << ", \"order_id\": \" " << order_id << "\" \"
    << ", \"symbol\": \" " << symbol << "\" \"
    << ", \"side\": \" " << side << "\" \"
    << ", \"order_type\": \" " << order_type << "\" \"
    << ", \"quantity\": " << quantity
    << ", \"fill_price\": " << fill_price
    << ", \"new_cash\": " << new_cash
    << ", \"new_quantity\": " << new_quantity
    << ", \"new_avg_price\": " << new_avg_price
    << ", \"timestamp\": " << time_stamp
    << "}\";

    std::lock_guard<std::mutex> guard(lock);
    file << ss.str();
    file.flush();
}

```

What to notice: the mutex guards concurrent writers, the JSON is hand-built with string concatenation (not a JSON library — fine for a fixed, controlled schema, but fragile if a string field ever contains a quote character), and `file.flush()` pushes the write to the OS immediately rather than waiting for a buffered flush — this is what makes the WAL durable across a process crash.

2d. The bridge — `wal_producer.py`

`pipeline/wal_producer.py`

```

producer = KafkaProducer(
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

position = load_cursor()
while True:
    if os.path.exists(WAL_PATH):
        with open(WAL_PATH, "r") as file:
            file.seek(position)
            while True:
                line = file.readline()
                if not line:
                    break
                if line.strip():
                    data = json.loads(line)
                    producer.send(KAFKA_TOPIC, value=data)
                    producer.flush()
                    save_cursor(file.tell())
                    position = file.tell()
            time.sleep(0.1)
    else:
        time.sleep(1)
        continue

```

What to notice:

- `load_cursor()` / `save_cursor()` persist a byte offset into `wal.log` in a sidecar file, `wal.cursor` — this is what makes the producer resumable across restarts.
- `producer.flush()` is called *after every single line*, and the cursor is only saved after that flush returns. That ordering is deliberate: it guarantees a line is confirmed in Kafka before the producer considers it "sent" and moves its cursor past it. The cost is throughput — one network round trip per fill, no batching.
- The loop re-opens and re-seeks the file every iteration rather than holding one open handle and polling — simple, but means every 0.1s tick pays a fresh `open()/seek()`.

2e. The read side — `fill_consumer.py`

`pipeline/fill_consumer.py`


```

consumer = KafkaConsumer(
    KAFKA_TOPIC,
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
    group_id=KAFKA_GROUP_ID,
    value_deserializer=lambda m: json.loads(m.decode("utf-8")),
    auto_offset_reset="earliest",
    enable_auto_commit=True
)
with app.app_context():
    for message in consumer:
        process_fill(message.value)

```

`process_fill()` then, per message: creates an `Order` row (`id` = the original `order_id`, status `FILLED`), overwrites `Account.cash_balance` with `new_cash`, and either updates an existing `Position` or inserts a new one — all in one `db.session.commit()`.

Why this design: it keeps Postgres writes off the order's hot path entirely. The HTTP request that placed the order returns as soon as the C++ engine approves and WAL-logs the fill; Postgres catches up asynchronously via this consumer. That's the "decoupling state sync from the HTTP request lifecycle" line from the README, made concrete.

2f. Infrastructure — `docker-compose.yml`

`docker-compose.yml`

```

kafka:
  image: apache/kafka:3.9.0
  ports:
    - "9092:9092"
  environment:
    KAFKA_NODE_ID: 0
    KAFKA_PROCESS_ROLES: controller,broker
    KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS: 0@kafka:9093
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_LOG_DIRS: /var/lib/kafka/data
  volumes:
    - kafka_data:/var/lib/kafka/data

```

Setting	What it means here
<code>KAFKA_PROCESS_ROLES: controller,broker</code>	Single node plays both KRaft roles — no separate controller cluster, no ZooKeeper
<code>KAFKA_LISTENERS</code> vs <code>ADVERTISED_LISTENERS</code>	Listeners are where the broker binds inside the container (<code>:9092/:9093</code>); advertised listeners are what it tells clients

	to connect back to (localhost:9092, since clients here run on the host, not inside the Compose network)
KAFKA_CONTROLLER_QUORUM_VOTERS: 0@kafka:9093	The KRaft quorum has exactly one voter — this node — at node ID 0
KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1	Consumer offsets themselves are stored with replication factor 1 — consistent with there being only one broker to replicate to

2g. Configuration — config.py

config.py

```
KAFKA_BOOTSTRAP_SERVERS = os.getenv("KAFKA_BOOTSTRAP_SERVERS", "localhost:9092")
KAFKA_TOPIC = os.getenv("KAFKA_TOPIC", "order-fills")
KAFKA_MARKET_TOPIC = os.getenv("KAFKA_MARKET_TOPIC", "market-ticks")
```

KAFKA_MARKET_TOPIC is defined but currently unused — see 2h.

2h. What Kafka does *not* touch

README drift: README.md's architecture diagram shows both `trade_filled` and prices flowing through Kafka to a WebSocket service, and lists Redpanda in the services table. That was true once — git history shows commit 531ec98 ("Added websocket connection via kafka") added a `binance_feed_producer.py` that published price ticks to Kafka. Commit 9574ff6 ("Added orderbook and gRPC connection") replaced it with `pipeline/orderbook_feed_producer.py`, which pushes Binance order-book snapshots **directly over gRPC** to the C++ engine — no Kafka involved. There is no `websocket_service.py` or `kafka_service.py` in the repo today despite being named in the README's file tree, and KAFKA_MARKET_TOPIC has no producer or consumer anywhere.

Practical takeaway: today, Kafka's footprint is *only* the order-fill pipeline (2a–2e). If describing this project, that's the accurate scope. Either update the README's diagram or treat it as a documented "future direction" rather than current behavior.

Part 3 — Honest Gaps (Interview Talking Points)

Being able to name these unprompted is a stronger signal than pretending the pipeline is bulletproof. Each of these is a legitimate, scoped next step — not a design failure for a portfolio project.

1. Redelivery isn't handled gracefully. The consumer auto-commits on an interval (`enable_auto_commit=True`), so a crash between processing a message and the next auto-commit replays it. `process_fill()` inserts an `Order` row with `id=order_id` as primary key — a true duplicate insert would raise an integrity error and **crash the consumer** rather than silently reprocessing. Combined with auto-commit, a crashing message becomes a "poison pill": the consumer restarts, replays the same offset range, hits the same crash, forever. There's no idempotency check (e.g. "does this `order_id` already exist?") before writing.

2. No dead-letter queue or error boundary. The consumer loop (for `message in consumer: process_fill(message.value)`) has no `try/except`. Any malformed message or DB error stops the whole consumer rather than being isolated and retried/parked.

3. No message key → no ordering guarantee across a user's fills. If a user places two orders in quick succession and (hypothetically) the topic ever had more than one partition, nothing pins both fills to the same partition, so processing order across them isn't guaranteed.

4. Single broker, replication factor 1. Fine for local dev/demo; explicitly not fault-tolerant. Worth saying so directly rather than being asked "what happens if the broker dies?" and improvising.

5. Producer flushes per-message. Durable, but throughput-limited by design (one round trip per fill) — a deliberate tradeoff worth naming rather than an oversight.

6. Ledger table is unpopulated. `ledger_entries` exists as a table and `Position.realised_pnl` exists as a column, but nothing in `fill_consumer.py` writes to either yet. Unrealized PnL is currently computed on-demand in `portfolio_routes.py` by calling the live risk engine price per request — not derived from the Kafka stream at all. This is exactly the gap Part 4 addresses.

Part 4 — Extending It: A PnL/Analytics Consumer

4a. Why a second consumer group is the right lever

Because consumer groups are independent (Part 1c), you can attach a brand-new consumer group to the existing `order-fills` topic without touching the producer, without touching `fill_consumer.py`, and without risking the `order-fill` → `Postgres` path at all. This is the actual payoff of having routed fills through Kafka instead of writing PnL logic straight into `OrderService._execute_fill` — a second, independent reader can be added, removed, or reset without any coordination with the first.

4b. Step-by-step implementation sketch

1. New file: `pipeline/pnl_consumer.py`, modeled directly on `fill_consumer.py`, but with its own group id, e.g. `KAFKA_GROUP_ID = os.getenv("KAFKA_PNL_GROUP_ID", "pnl-analytics-consumer-group")`. Same topic (`order-fills`), same bootstrap servers.
2. On each message, branch on side:
 - **BUY** — no realized PnL yet; a buy only changes cost basis (which the risk engine already computed as `new_avg_price`). Nothing to realize.
 - **SELL** — realized PnL on *this fill* is $(\text{fill_price} - \text{average_price_before_this_fill}) * \text{quantity}$. Note: the message already carries `new_avg_price` (post-fill), not the pre-fill average — you need the position's average price *before* applying this fill's quantity change, so either read `Position.average_price` from the DB before `fill_consumer.py`'s update lands, or (safer) have the WAL entry carry the pre-fill average price explicitly rather than re-deriving it. Worth flagging as a real design decision, not an afterthought.
3. Write the result as an append-only `LedgerEntry` (entry_type e.g. `"REALIZED_PNL"`, amount = the computed delta, linked via `order_id`) and add that delta onto `Position.realised_pnl`.
4. Run it as its own process (own Docker service / own terminal), exactly parallel to how `fill_consumer.py` is run today — it's just another consumer group on the same topic.

4c. Idempotency & redelivery safety

Don't repeat Gap #1 from Part 3 in the new consumer. Since `LedgerEntry.order_id` already exists as a column, guard the write: before inserting a `REALIZED_PNL` entry, check whether one already exists for this `order_id`, and skip if so. That makes the PnL consumer safe under at-least-once

redelivery even though the underlying pipeline doesn't otherwise enforce exactly-once — the idempotency lives in the consumer, which is the standard place to put it.

4d. Backfilling history

Because it's a new consumer group, setting `auto_offset_reset="earliest"` lets it replay the entire topic from the start the first time it runs — useful for backfilling PnL for fills that happened before this consumer existed. The caveat: Kafka topics have finite retention, so this only works for fills still within the topic's retention window. For a complete historical backfill beyond that window, the source of truth is the orders table in Postgres, not Kafka — Kafka is a transport/decoupling layer here, not the system of record.

Part 5 — Hands-On: Inspecting Kafka Locally

The `apache/kafka:3.9.0` image ships the standard Kafka CLI tools under `/opt/kafka/bin/`. With the stack up (`docker compose up -d`), run these via `docker compose exec kafka ...`:

```
# List topics
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --list

# Describe the order-fills topic (partition count, replicas, leader)
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:9092 --describe --topic order-fills

# Tail messages live (good for watching wal_producer.py in action)
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server localhost:9092 --topic order-fills --from-beginning

# Check a consumer group's lag (are fill_consumer.py or a new PnL consumer keeping up?)
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --group order-fill-consumer-group
```

The consumer-group lag command is the most useful one to know cold: it shows current offset, log-end offset, and the gap (lag) per partition — the standard way to answer "is this consumer keeping up with production?"

Part 6 — The 60-Second Elevator Pitch

"When an order fills, the C++ risk engine appends the fill to a local write-ahead log rather than calling Kafka directly — that keeps the hot path fast and durable even if Kafka is unavailable. A separate producer process tails that WAL file, using a persisted byte-offset cursor so it can resume after a restart, and republishes each fill onto a Kafka topic. A consumer in its own consumer group picks those events up asynchronously and applies them to Postgres — updating orders, cash balance, and positions — completely off the HTTP request path. It's an at-least-once pipeline: durable and decoupled, though it doesn't yet defend against duplicate redelivery or isolate bad messages, which are the next things I'd harden. The same decoupling is also why adding new consumers — like a PnL/analytics engine — doesn't require touching the existing fill-processing path at all; it's just another independent reader on the same topic."